

On the other hand, some of my colleagues at Naughty Dog have found that a good SLERP implementation performs nearly as well as LERP. (For example, on the PS3's SPUs, Naughty Dog's Ice team's implementation of SLERP takes 20 cycles per joint, while its LERP implementation takes 16.25 cycles per joint.) Therefore, I'd personally recommend that you profile your SLERP and LERP implementations before making any decisions. If the performance hit for SLERP isn't unacceptable, I say go for it, because it may result in slightly better-looking animations. But if your SLERP is slow (and you cannot speed it up, or you just don't have the time to do so), then LERP is usually good enough for most purposes.

5.5 Comparison of Rotational Representations

We've seen that rotations can be represented in quite a few different ways. This section summarizes the most common rotational representations and outlines their pros and cons. No one representation is ideal in all situations. Using the information in this section, you should be able to select the best representation for a particular application.

5.5.1 Euler Angles

We briefly explored *Euler angles* in Section 5.3.9.1. A rotation represented via Euler angles consists of three scalar values: yaw, pitch and roll. These quantities are sometimes represented by a 3D vector $[\theta_Y \ \theta_P \ \theta_R]$.

The benefits of this representation are its simplicity, its small size (three floating-point numbers) and its intuitive nature—yaw, pitch and roll are easy to visualize. You can also easily interpolate simple rotations about a single axis. For example, it's trivial to find intermediate rotations between two distinct yaw angles by linearly interpolating the scalar θ_Y . However, Euler angles cannot be interpolated easily when the rotation is about an arbitrarily oriented axis.

In addition, Euler angles are prone to a condition known as *gimbal lock*. This occurs when a 90-degree rotation causes one of the three principal axes to “collapse” onto another principal axis. For example, if you rotate by 90 degrees about the x -axis, the y -axis collapses onto the z -axis. This prevents any further rotations about the original y -axis, because rotations about y and z have effectively become equivalent.

Another problem with Euler angles is that the order in which the rotations are performed around each axis matters. The order could be PYR, YPR, RYP and so on, and each ordering may produce a different composite rotation. No

one standard rotation order exists for Euler angles across all disciplines (although certain disciplines do follow specific conventions). So the rotation angles $[\theta_Y \ \theta_P \ \theta_R]$ do not uniquely define a particular rotation—you need to know the rotation order to interpret these numbers properly.

A final problem with Euler angles is that they depend upon the mapping from the x -, y - and z -axes onto the natural *front*, *left/right* and *up* directions for the object being rotated. For example, yaw is always defined as rotation about the *up* axis, but without additional information we cannot tell whether this corresponds to a rotation about x , y or z .

5.5.2 3×3 Matrices

A 3×3 matrix is a convenient and effective rotational representation for a number of reasons. It does not suffer from gimbal lock, and it can represent arbitrary rotations uniquely. Rotations can be applied to points and vectors in a straightforward manner via matrix multiplication (i.e., a series of dot products). Most CPUs and all GPUs now have built-in support for hardware-accelerated dot products and matrix multiplication. Rotations can also be reversed by finding an inverse matrix, which for a pure rotation matrix is the same thing as finding the transpose—a trivial operation. And 4×4 matrices offer a way to represent arbitrary affine transformations—rotations, translations and scaling—in a totally consistent way.

However, rotation matrices are not particularly intuitive. Looking at a big table of numbers doesn't help one picture the corresponding transformation in three-dimensional space. Also, rotation matrices are not easily interpolated. Finally, a rotation matrix takes up a lot of storage (nine floating-point numbers) relative to Euler angles (three floats).

5.5.3 Axis + Angle

We can represent rotations as a unit vector, defining the axis of rotation plus a scalar for the angle of rotation. This is known as an *axis+angle* representation, and it is sometimes denoted by the four-dimensional vector $[\mathbf{a} \ \theta] = [a_x \ a_y \ a_z \ \theta]$, where $\mathbf{a}$ is the axis of rotation and θ the angle in radians. In a right-handed coordinate system, the direction of a positive rotation is defined by the right-hand rule, while in a left-handed system, we use the left-hand rule instead.

The benefits of the axis+angle representation are that it is reasonably intuitive and also compact. (It only requires four floating-point numbers, as opposed to the nine required for a 3×3 matrix.)

One important limitation of the axis+angle representation is that rotations cannot be easily interpolated. Also, rotations in this format cannot be applied to points and vectors in a straightforward way—one needs to convert the axis+angle representation into a matrix or quaternion first.

5.5.4 Quaternions

As we've seen, a unit-length quaternion can represent 3D rotations in a manner analogous to the axis+angle representation. The primary difference between the two representations is that a quaternion's axis of rotation is scaled by the sine of the half-angle of rotation, and instead of storing the angle in the fourth component of the vector, we store the cosine of the half-angle.

The quaternion formulation provides two immense benefits over the axis+angle representation. First, it permits rotations to be concatenated and applied directly to points and vectors via quaternion multiplication. Second, it permits rotations to be easily interpolated via simple LERP or SLERP operations. Its small size (four floating-point numbers) is also a benefit over the matrix formulation.

5.5.5 SRT Transformations

By itself, a quaternion can only represent a rotation, whereas a 4×4 matrix can represent an arbitrary affine transformation (rotation, translation and scale). When a quaternion is combined with a *translation vector* and a *scale factor* (either a scalar for uniform scaling or a vector for nonuniform scaling), then we have a viable alternative to the 4×4 matrix representation of affine transformations. We sometimes call this an *SRT transform*, because it contains a scale factor, a rotation quaternion and a translation vector. (It's also sometimes called an *SQT*, because the rotation is a quaternion.)

$$\text{SRT} = [s \quad \mathbf{q} \quad \mathbf{t}] \quad (\text{uniform scale } s),$$

or

$$\text{SRT} = [s \quad \mathbf{q} \quad \mathbf{t}] \quad (\text{nonuniform scale vector } \mathbf{s}).$$

SRT transforms are widely used in computer animation because of their smaller size (eight floats for uniform scale, or ten floats for nonuniform scale, as opposed to the 12 floating-point numbers needed for a 4×3 matrix) and their ability to be easily interpolated. The translation vector and scale factor are interpolated via LERP, and the quaternion can be interpolated with either LERP or SLERP.

5.5.6 Dual Quaternions

A *rigid transformation* is a transformation involving a rotation and a translation—a “corkscrew” motion. Such transformations are prevalent in animation and robotics. A rigid transformation can be represented using a mathematical object known as a *dual quaternion*. The dual quaternion representation offers a number of benefits over the typical vector-quaternion representation. The key benefit is that linear interpolation blending can be performed in a constant-speed, shortest-path, coordinate-invariant manner, similar to using LERP for translation vectors and SLERP for rotational quaternions (see Section 5.4.5.1), but in a way that is easily generalizable to blends involving three or more transforms.

A dual quaternion is like an ordinary quaternion, except that its four components are *dual numbers* instead of regular real-valued numbers. A dual number can be written as the sum of a non-dual part and a dual part as follows: $\hat{a} = a + \varepsilon b$. Here ε is a magical number called the *dual unit*, defined in such a way that $\varepsilon^2 = 0$ (yet without ε itself being zero). This is analogous to the imaginary number $j = \sqrt{-1}$ used when writing a complex number as the sum of a real and an imaginary part: $c = a + jb$.

Because each dual number can be represented by two real numbers (the non-dual and dual parts, a and b), a dual quaternion can be represented by an eight-element vector. It can also be represented as the sum of two ordinary quaternions, where the second one is multiplied by the dual unit, as follows: $\hat{q} = q_a + \varepsilon q_b$.

A full discussion of dual numbers and dual quaternions is beyond our scope here. However, the excellent paper entitled, “Dual Quaternions for Rigid Transformation Blending” by Kavan et al. outlines the theory and practice of using dual quaternions to represent rigid transformations—it is available online at <https://bit.ly/2vjD5sz>. Note that in this paper, a dual number is written in the form $\hat{a} = a_0 + \varepsilon a_\varepsilon$, whereas I have used $a + \varepsilon b$ above to underscore the similarity between dual numbers and complex numbers.¹

5.5.7 Rotations and Degrees of Freedom

The term “*degrees of freedom*” (or *DOF* for short) refers to the number of mutually independent ways in which an object’s physical state (position and orientation) can change. You may have encountered the phrase “six degrees of

¹Personally I would have preferred the symbol a_1 over a_0 , so that a dual number would be written $\hat{a} = (1)a_1 + (\varepsilon)a_\varepsilon$. Just as when we plot a complex number in the complex plane, we can think of the real unit as a “basis vector” along the real axis, and the dual unit ε as a “basis vector” along the dual axis.

freedom” in fields such as mechanics, robotics and aeronautics. This refers to the fact that a three-dimensional object (whose motion is not artificially constrained) has three degrees of freedom in its translation (along the x -, y - and z -axes) and three degrees of freedom in its rotation (about the x -, y - and z -axes), for a total of six degrees of freedom.

The DOF concept will help us to understand how different rotational representations can employ different numbers of floating-point parameters, yet all specify rotations with only three degrees of freedom. For example, Euler angles require three floats, but axis+angle and quaternion representations use four floats, and a 3×3 matrix takes up nine floats. How can these representations all describe 3-DOF rotations?

The answer lies in *constraints*. All 3D rotational representations employ three or more floating-point parameters, but some representations also have one or more constraints on those parameters. The constraints indicate that the parameters are not *independent*—a change to one parameter induces changes to the other parameters in order to maintain the validity of the constraint(s). If we subtract the number of constraints from the number of floating-point parameters, we arrive at the number of degrees of freedom—and this number should always be three for a 3D rotation:

$$N_{\text{DOF}} = N_{\text{parameters}} - N_{\text{constraints}}. \quad (5.10)$$

The following list shows Equation (5.10) in action for each of the rotational representations we’ve encountered in this book.

- *Euler Angles*. 3 parameters – 0 constraints = 3 DOF.
- *Axis+Angle*. 4 parameters – 1 constraint = 3 DOF.
Constraint: Axis is constrained to be unit length.
- *Quaternion*. 4 parameters – 1 constraint = 3 DOF.
Constraint: Quaternion is constrained to be unit length.
- *3×3 Matrix*. 9 parameters – 6 constraints = 3 DOF.
Constraints: All three rows and all three columns must be of unit length (when treated as three-element vectors).

5.6 Other Useful Mathematical Objects

As game engineers, we will encounter a host of other mathematical objects in addition to points, vectors, matrices and quaternions. This section briefly outlines the most common of these.

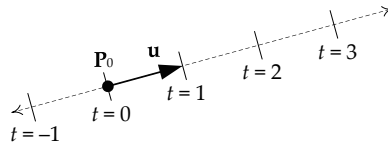


Figure 5.25. Parametric equation of a line.

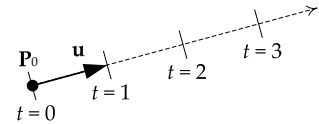


Figure 5.26. Parametric equation of a ray.

5.6.1 Lines, Rays and Line Segments

An infinite line can be represented by a point $\mathbf{P}_0$ plus a unit vector $\mathbf{u}$ in the direction of the line. A *parametric equation* of a line traces out every possible point $\mathbf{P}$ along the line by starting at the initial point $\mathbf{P}_0$ and moving an arbitrary distance t along the direction of the unit vector $\mathbf{v}$. The infinitely large set of points $\mathbf{P}$ becomes a *vector function* of the scalar parameter t :

$$\mathbf{P}(t) = \mathbf{P}_0 + t\mathbf{u}, \quad \text{where } -\infty < t < \infty. \quad (5.11)$$

This is depicted in Figure 5.25.

A *ray* is a line that extends to infinity in only one direction. This is easily expressed as $\mathbf{P}(t)$ with the constraint $t \geq 0$, as shown in Figure 5.26.

A *line segment* is bounded at both ends by $\mathbf{P}_0$ and $\mathbf{P}_1$. It too can be represented by $\mathbf{P}(t)$, in either one of the following two ways (where $\mathbf{L} = \mathbf{P}_1 - \mathbf{P}_0$, $L = |\mathbf{L}|$ is the length of the line segment, and $\mathbf{u} = (1/L)\mathbf{L}$ is a unit vector in the direction of $\mathbf{L}$):

1. $\mathbf{P}(t) = \mathbf{P}_0 + t\mathbf{u}$, where $0 \leq t \leq L$, or
2. $\mathbf{P}(t) = \mathbf{P}_0 + t\mathbf{L}$, where $0 \leq t \leq 1$.

The latter format, depicted in Figure 5.27, is particularly convenient because the parameter t is normalized; in other words, t always goes from zero to one, no matter which particular line segment we are dealing with. This means we do not have to store the constraint L in a separate floating-point parameter; it is already encoded in the vector $\mathbf{L} = L\mathbf{u}$ (which we have to store anyway).

5.6.2 Spheres

Spheres are ubiquitous in game engine programming. A sphere is typically defined as a center point $\mathbf{C}$ plus a radius r , as shown in Figure 5.28. This packs nicely into a four-element vector, $[C_x \ C_y \ C_z \ r]$. As we saw when we discussed SIMD vector processing, there are distinct benefits to being able to pack data into a vector containing four 32-bit floats (i.e., a 128-bit package).

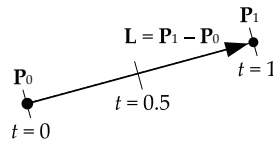
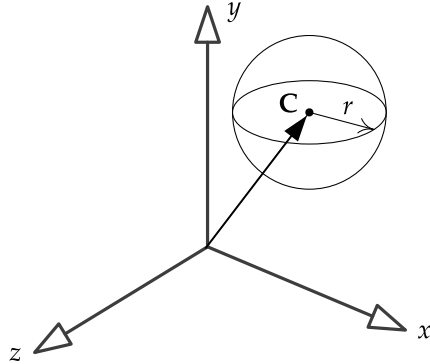
Figure 5.27. Parametric equation of a line segment, with normalized parameter t .

Figure 5.28. Point-radius representation of a sphere.

5.6.3 Planes

A *plane* is a 2D surface in 3D space. As you may recall from high-school algebra, the equation of a plane is often written as follows:

$$Ax + By + Cz + D = 0.$$

This equation is satisfied only for the locus of points $\mathbf{P} = [x \ y \ z]$ that lie on the plane.

Planes can be represented by a point $\mathbf{P}_0$ and a unit vector $\mathbf{n}$ that is normal to the plane. This is sometimes called *point-normal form*, as depicted in Figure 5.29.

It's interesting to note that when the parameters A , B and C from the traditional plane equation are interpreted as a 3D vector, that vector lies in the direction of the plane normal. If the vector $[A \ B \ C]$ is normalized to unit length, then the normalized vector $[a \ b \ c] = \mathbf{n}$, and the normalized parameter $d = D / \sqrt{A^2 + B^2 + C^2}$ is just the distance from the plane to the origin. The sign of d is positive if the plane's normal vector $\mathbf{n}$ is pointing toward the origin (i.e., the origin is on the "front" side of the plane) and negative if the normal is pointing away from the origin (i.e., the origin is "behind" the plane).

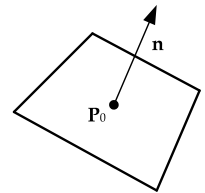


Figure 5.29. A plane in point-normal form.

Another way of looking at this is that the plane equation and the point-normal form are really just two ways of writing the same equation. Imagine testing whether or not an arbitrary point $\mathbf{P} = [x \ y \ z]$ lies on the plane. To do this, we find the signed distance from point $\mathbf{P}$ to the origin along the normal $\mathbf{n} = [a \ b \ c]$, and if this signed distance is equal to the signed distance $d = -\mathbf{n} \cdot \mathbf{P}_0$ from the plane from the origin, then $\mathbf{P}$ must lie on the plane. So let's set them equal and expand some terms:

$$\begin{aligned}
 (\text{signed distance } \mathbf{P} \text{ to origin}) &= (\text{signed distance plane to origin}) \\
 \mathbf{n} \cdot \mathbf{P} &= \mathbf{n} \cdot \mathbf{P}_0 \\
 \mathbf{n} \cdot \mathbf{P} - \mathbf{n} \cdot \mathbf{P}_0 &= 0 \\
 ax + by + cz - \mathbf{n} \cdot \mathbf{P}_0 &= 0 \\
 ax + by + cz + d &= 0.
 \end{aligned} \tag{5.12}$$

Equation (5.12) only holds when the point $\mathbf{P}$ lies on the plane. But what happens when the point $\mathbf{P}$ does *not* lie on the plane? In this case, the left-hand side of the plane equation ($ax + by + cz$, which is equal to $\mathbf{n} \cdot \mathbf{P}$) tells how far “off” the point is from being on the plane. This expression calculates the difference between the distance from $\mathbf{P}$ to the origin and the distance from the plane to the origin. In other words, the left-hand side of Equation (5.12) gives us the perpendicular distance h between the point and the plane! This is just another way to write Equation (5.2) from Section 5.2.4.7.

$$\begin{aligned}
 h &= (\mathbf{P} - \mathbf{P}_0) \cdot \mathbf{n}; \\
 h &= ax + by + cz + d.
 \end{aligned} \tag{5.13}$$

A plane can actually be packed into a four-element vector, much like a sphere can. To do so, we observe that to describe a plane uniquely, we need only the normal vector $\mathbf{n} = [a \ b \ c]$ and the distance from the origin d . The four-element vector $\mathbf{L} = [\mathbf{n} \ d] = [a \ b \ c \ d]$ is a compact and convenient way to represent and store a plane in memory. Note that when $\mathbf{P}$ is written in homogeneous coordinates with $w = 1$, the equation $(\mathbf{L} \cdot \mathbf{P}) = 0$ is yet another way of writing $(\mathbf{n} \cdot \mathbf{P}) = -d$. These equations are satisfied for all points $\mathbf{P}$ that lie on the plane $\mathbf{L}$.

Planes defined in four-element vector form can be easily transformed from one coordinate space to another. Given a matrix $\mathbf{M}_{A \rightarrow B}$ that transforms points and (non-normal) vectors from space A to space B, we already know that to transform a *normal vector* such as the plane's $\mathbf{n}$ vector, we need to use the inverse transpose of that matrix, $(\mathbf{M}_{A \rightarrow B}^{-1})^T$. So it shouldn't be a big surprise to learn that applying the inverse transpose of a matrix to a four-element plane vector $\mathbf{L}$ will, in fact, correctly transform that plane from space A to space B. We won't derive or prove this result any further here, but a thorough explanation of why this little “trick” works is provided in Section 4.2.3 of [32].

5.6.4 Axis-Aligned Bounding Boxes (AABB)

An axis-aligned bounding box (AABB) is a 3D *cuboid* whose six rectangular faces are aligned with a particular coordinate frame's mutually orthogonal axes. As such, an AABB can be represented by a six-element vector containing the minimum and maximum coordinates along each of the 3 principal axes, $[x_{\min}, y_{\min}, z_{\min}, x_{\max}, y_{\max}, z_{\max}]$, or two points $\mathbf{P}_{\min}$ and $\mathbf{P}_{\max}$.

This simple representation allows for a particularly convenient and inexpensive method of testing whether a point $\mathbf{P}$ is inside or outside any given AABB. We simply test if all of the following conditions are true:

$$\begin{aligned} P_x &\geq x_{\min} \text{ and } P_x \leq x_{\max} \text{ and} \\ P_y &\geq y_{\min} \text{ and } P_y \leq y_{\max} \text{ and} \\ P_z &\geq z_{\min} \text{ and } P_z \leq z_{\max}. \end{aligned}$$

Because intersection tests are so speedy, AABBs are often used as an “early out” collision check; if the AABBs of two objects do not intersect, then there is no need to do a more detailed (and more expensive) collision test.

5.6.5 Oriented Bounding Boxes (OBB)

An oriented bounding box (OBB) is a *cuboid* that has been oriented so as to align in some logical way with the object it bounds. Usually an OBB aligns with the local-space axes of the object. Hence, it acts like an AABB in local space, although it may not necessarily align with the world-space axes.

Various techniques exist for testing whether or not a point lies within an OBB, but one common approach is to transform the point into the OBB's “aligned” coordinate system and then use an AABB intersection test as presented above.

5.6.6 Frusta

As shown in Figure 5.30, a *frustum* is a group of six planes that define a *truncated pyramid* shape. Frusta are commonplace in 3D rendering because they conveniently define the viewable region of the 3D world when rendered via a perspective projection from the point of view of a virtual camera. Four of the planes bound the edges of the screen space, while the other two planes represent the the near and far clipping planes (i.e., they define the minimum and maximum z coordinates possible for any visible point).

One convenient representation of a frustum is as an array of six planes, each of which is represented in point-normal form (i.e., one point and one normal vector per plane).

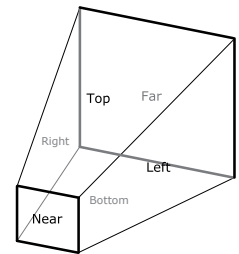


Figure 5.30. A frustum.

Testing whether a point lies inside a frustum is a bit involved, but the basic idea is to use dot products to determine whether the point lies on the front or back side of each plane. If it lies inside all six planes, it is inside the frustum.

A helpful trick is to transform the world-space point being tested by applying the camera's perspective projection to it. This takes the point from world space into a space known as *homogeneous clip space*. In this space, the frustum is just an axis-aligned cuboid (AABB). This permits much simpler in/out tests to be performed.

5.6.7 Convex Polyhedral Regions

A *convex polyhedral region* is defined by an arbitrary set of planes, all with normals pointing inward (or outward). The test for whether a point lies inside or outside the volume defined by the planes is relatively straightforward; it is similar to a frustum test, but with possibly more planes. Convex regions are very useful for implementing arbitrarily shaped trigger regions in games. Many engines employ this technique; for example, the Quake engine's ubiquitous *brushes* are just volumes bounded by planes in exactly this way.

5.7 Random Number Generation

Random numbers are ubiquitous in game engines, so it behooves us to have a brief look at the two most common random number generators (RNG), the linear congruential generator and the Mersenne Twister. It's important to realize that random number generators don't actually generate random numbers—they merely produce a complex, but totally deterministic, predefined sequence of values. For this reason, we call the sequences they produce *pseudorandom*, and technically speaking we should really call them "pseudorandom number generators" (PRNG). What differentiates a good generator from a bad one is how long the sequence of numbers is before it repeats (its *period*), and how well the sequences hold up under various well-known randomness tests.

5.7.1 Linear Congruential Generators

Linear congruential generators are a very fast and simple way to generate a sequence of pseudorandom numbers. Depending on the platform, this algorithm is sometimes used in the C standard library's `rand()` function. However, your mileage may vary, so don't count on `rand()` being based on any particular algorithm. If you want to be sure, you'll be better off implementing your own random number generator.

The linear congruential algorithm is explained in detail in the book *Numerical Recipes in C*, so I won't go into the details of it here.

What I will say is that this random number generator does not produce particularly high-quality pseudorandom sequences. Given the same initial seed value, the sequence is always exactly the same. The numbers produced do not meet many of the criteria widely accepted as desirable, such as a long period, low- and high-order bits that have similarly long periods, and absence of sequential or spatial correlation between the generated values.

5.7.2 Mersenne Twister

The Mersenne Twister pseudorandom number generator algorithm was designed specifically to improve upon the various problems of the linear congruential algorithm. Wikipedia provides the following description of the benefits of the algorithm:

1. It was designed to have a colossal period of $2^{19937} - 1$ (the creators of the algorithm proved this property). In practice, there is little reason to use larger ones, as most applications do not require 2^{19937} unique combinations ($2^{19937} \approx 4.3 \times 10^{6001}$).
2. It has a very high order of dimensional equidistribution. Note that this means, by default, that there is negligible serial correlation between successive values in the output sequence.
3. It passes numerous tests for statistical randomness, including the stringent Diehard tests.
4. It is fast.

Various implementations of the Twister are available on the web, including a particularly cool one that uses SIMD vector instructions for an extra speed boost, called *SFMT* (SIMD-oriented fast Mersenne Twister). SFMT can be downloaded from <http://www.math.sci.hiroshima-u.ac.jp/~m-mat/MT/SFMT/index.html>.

5.7.3 Mother-of-All, Xorshift and KISS99

In 1994, George Marsaglia, a computer scientist and mathematician best known for developing the Diehard battery of tests of randomness (<http://www.stat.fsu.edu/pub/diehard>), published a pseudorandom number generation algorithm that is much simpler to implement and runs faster than the Mersenne Twister algorithm. He claimed that it could produce a sequence

of 32-bit pseudorandom numbers with a period of non-repetition of 2^{250} . It passed all of the Diehard tests and still stands today as one of the best pseudorandom number generators for high-speed applications. He called his algorithm the *Mother of All Pseudorandom Number Generators*, because it seemed to him to be the only random number generator one would ever need.

Later, Marsaglia published another generator called Xorshift, which is between Mersenne and Mother-of-All in terms of randomness, but runs slightly faster than Mother.

Marsaglia also developed a series of random number generators that are collectively called KISS (Keep It Simple Stupid). The KISS99 algorithm is a popular choice, because it has a large period (2^{123}) and passes all tests in the TestU01 test suite (<https://bit.ly/2r5FmSP>).

You can read about George Marsaglia at http://en.wikipedia.org/wiki/George_Marsaglia, and about the Mother-of-All generator at <ftp://ftp.forth.org/pub/C/mother.c> and at <http://www.agner.org/random>. You can download a PDF of George's paper on Xorshift at <http://www.jstatsoft.org/v08/i14/paper>.

5.7.4 PCG

Another very popular and high-quality family of pseudorandom number generators is called PCG. It works by combining a congruential generator for its state transitions (the "CG" in PCG) with permutation functions to generate its output (the "P" in PCG). You can read more about this family of PRNGs at <http://www.pcg-random.org/>.

Part II

Low-Level

Engine Systems



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

6

Engine Support Systems

Every game engine requires some low-level support systems that manage mundane but crucial tasks, such as starting up and shutting down the engine, configuring engine and game features, managing the engine's memory usage, handling access to file system(s), providing access to the wide range of heterogeneous asset types used by the game (meshes, textures, animations, audio, etc.), and providing debugging tools for use by the game development team. This chapter will focus on the lowest-level support systems found in most game engines. In the chapters that follow, we will explore some of the larger core systems, including resource management, human interface devices and in-game debugging tools.

6.1 Subsystem Start-Up and Shut-Down

A game engine is a complex piece of software consisting of many interacting subsystems. When the engine first starts up, each subsystem must be configured and initialized in a specific order. Interdependencies between subsystems implicitly define the order in which they must be started—i.e., if subsystem B depends on subsystem A, then A will need to be started up before B can be initialized. Shut-down typically occurs in the reverse order, so B would shut down first, followed by A.